Deploying Metabase on AWS Fargate (ECS Service)

by Mohamad Ikhsan Nurulloh

AWS Fargate

Important Disclaimer - Before You Dive In

- **No Endorsement:** This guide is for educational purposes only and does not imply endorsement by Metabase or AWS.
- **Testing Only:** Intended for testing and development, not production environments.
- **Refer to Official Docs:** Always consult official Metabase and AWS documentation for production use.
- **Proceed at Your Own Risk:** Use this guide at your own risk and be aware of potential deployment implications.

AWS Fargate in a Nutshell

AWS Fargate

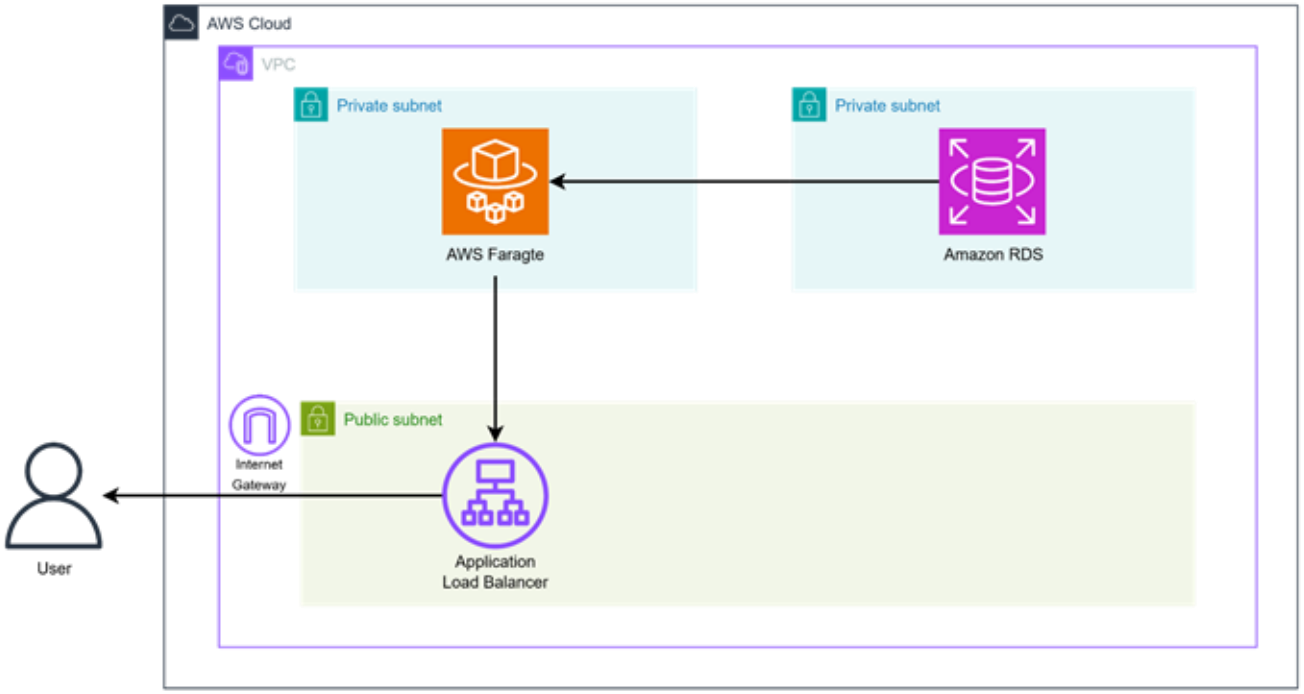
AWS Fargate is a technology that you can use with Amazon ECS to run containers without having to manage servers or clusters of Amazon EC2 instances. With AWS Fargate, you no longer have to provision, configure, or scale clusters of virtual machines to run containers. This removes the need to choose server types, decide when to scale your clusters, or optimize cluster packing.

Why Fargate for Metabase?

AWS Fargate

Deploying Metabase on Amazon ECS Fargate is a simple and low-cost way to run containers without managing servers. It removes the need for EC2 instances and load balancers, scales easily and reduces operational work, ideal for small to medium deployments or development environments.

High-Level Architecture

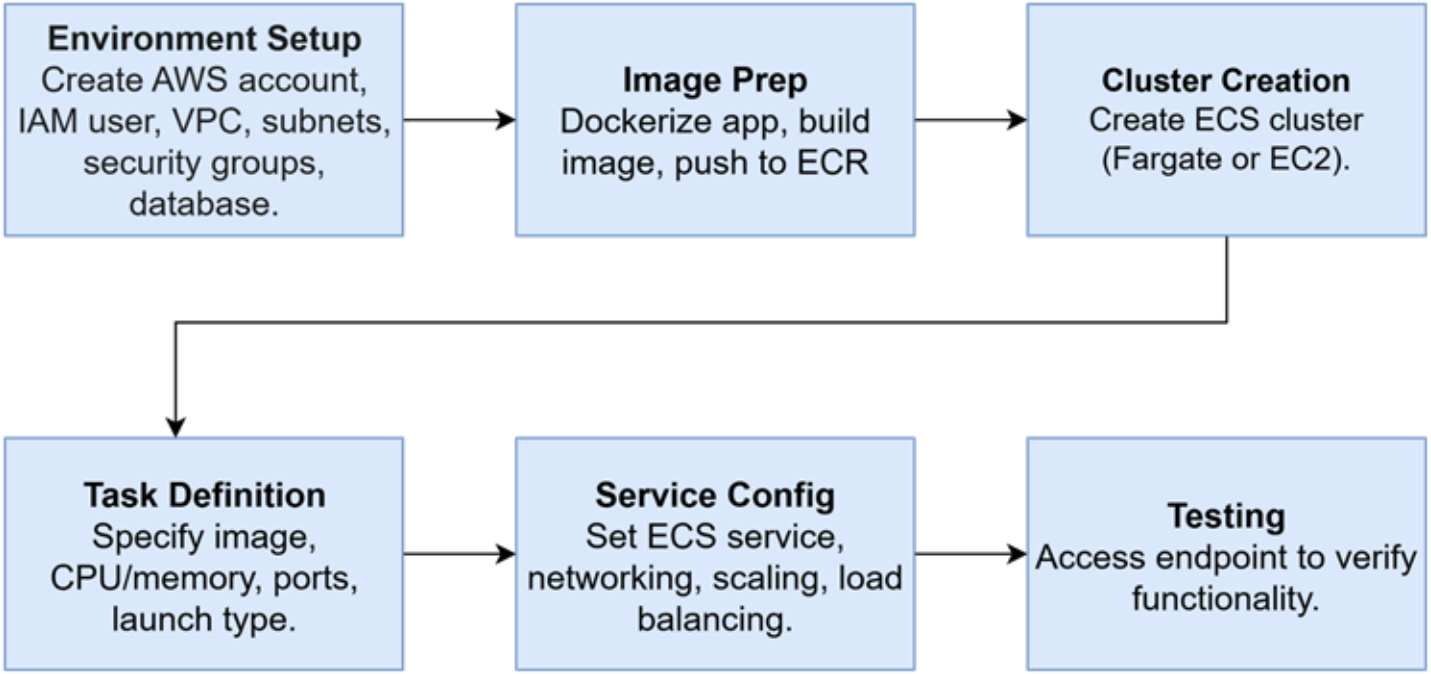


Overall Architecture: Metabase runs on AWS Fargate behind an Application Load Balancer (ALB) in a public subnet, which handles HTTPS traffic from users. The Metabase containers in private subnets store metadata in Amazon RDS. Security groups restrict traffic so only the ALB can reach Fargate and only Fargate can access RDS, delivering a secure, scalable, and fully managed analytics platform.

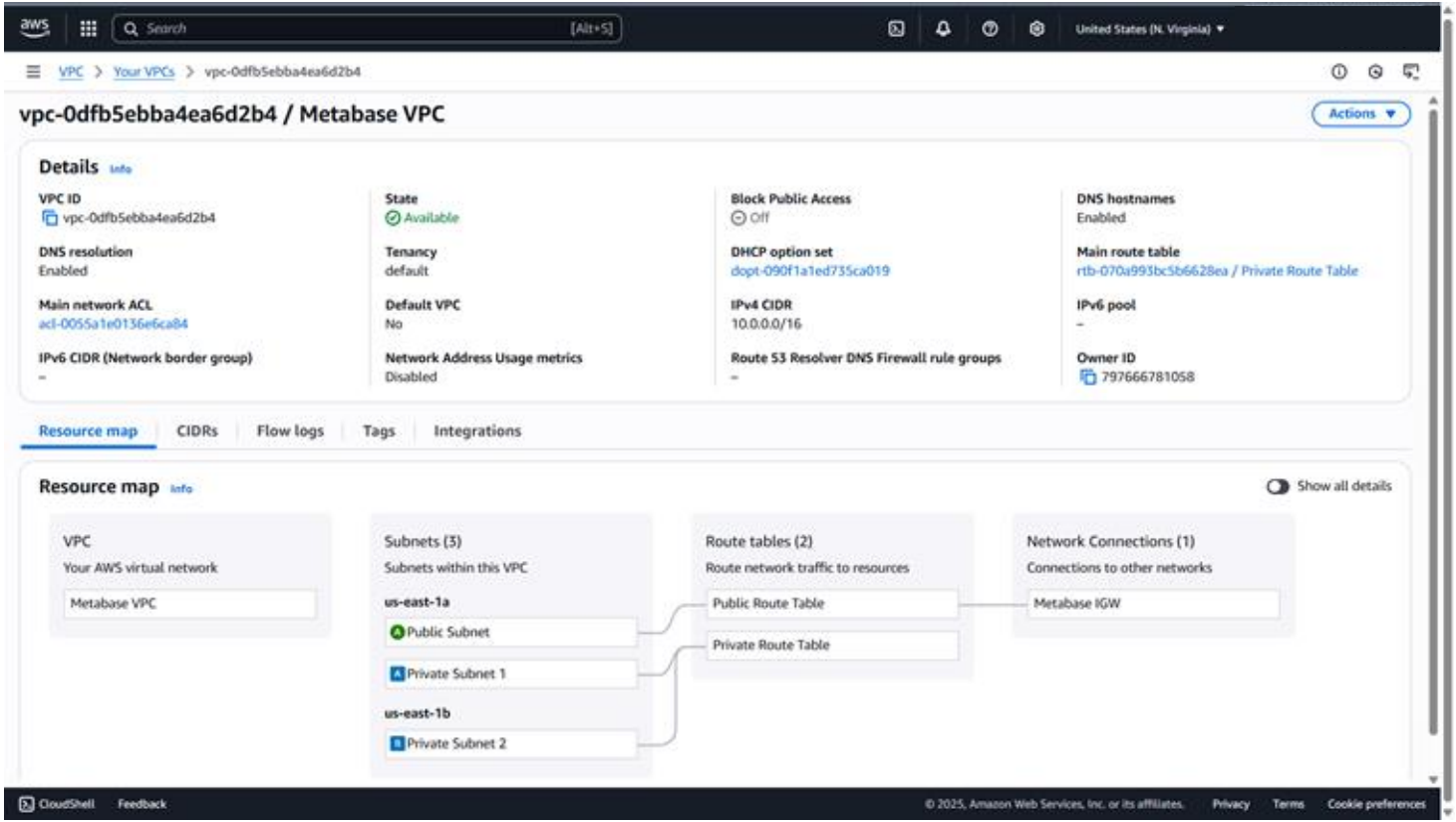
Inside the Architecture - Key Components Explained

- **VPC with Public & Private Subnets:** Public subnet hosts the ALB; private subnets host Fargate tasks and Amazon RDS for isolation and high availability.
- **Application Load Balancer (ALB):** Handles HTTPS from users via the Internet Gateway and securely routes requests to Fargate tasks.
- **AWS Fargate (ECS Service):** Runs Metabase containers without server management, scaling automatically to match demand. Outbound internet access is not available.
- **Amazon RDS:** Provides a managed PostgreSQL database in private subnets for storing all Metabase metadata and configuration.
- **Security:** Security groups restrict traffic so only ALB → Fargate → RDS communication is allowed, ensuring a highly controlled and secure network flow.

Deployment Roadmap

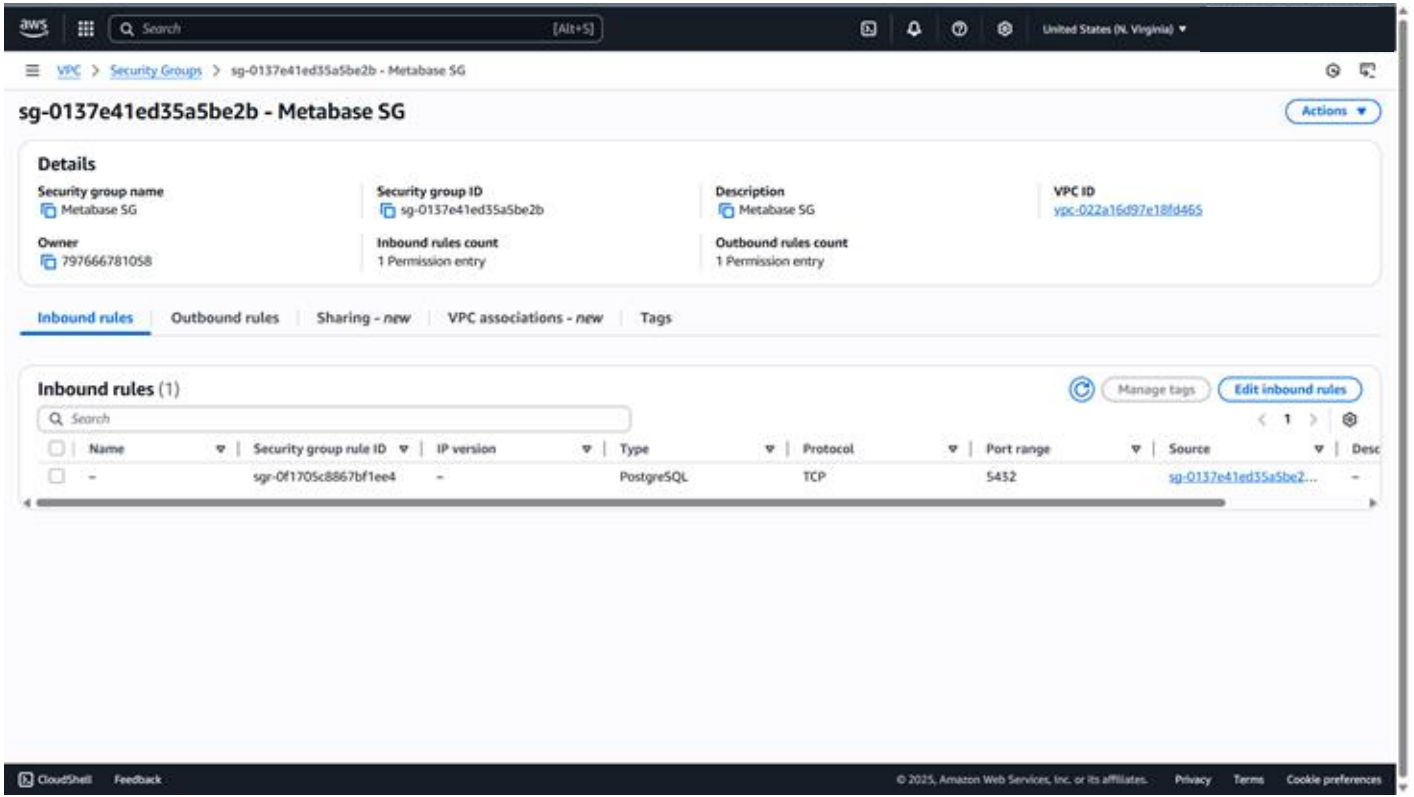


Building the Foundation: Network Infrastructure Setup



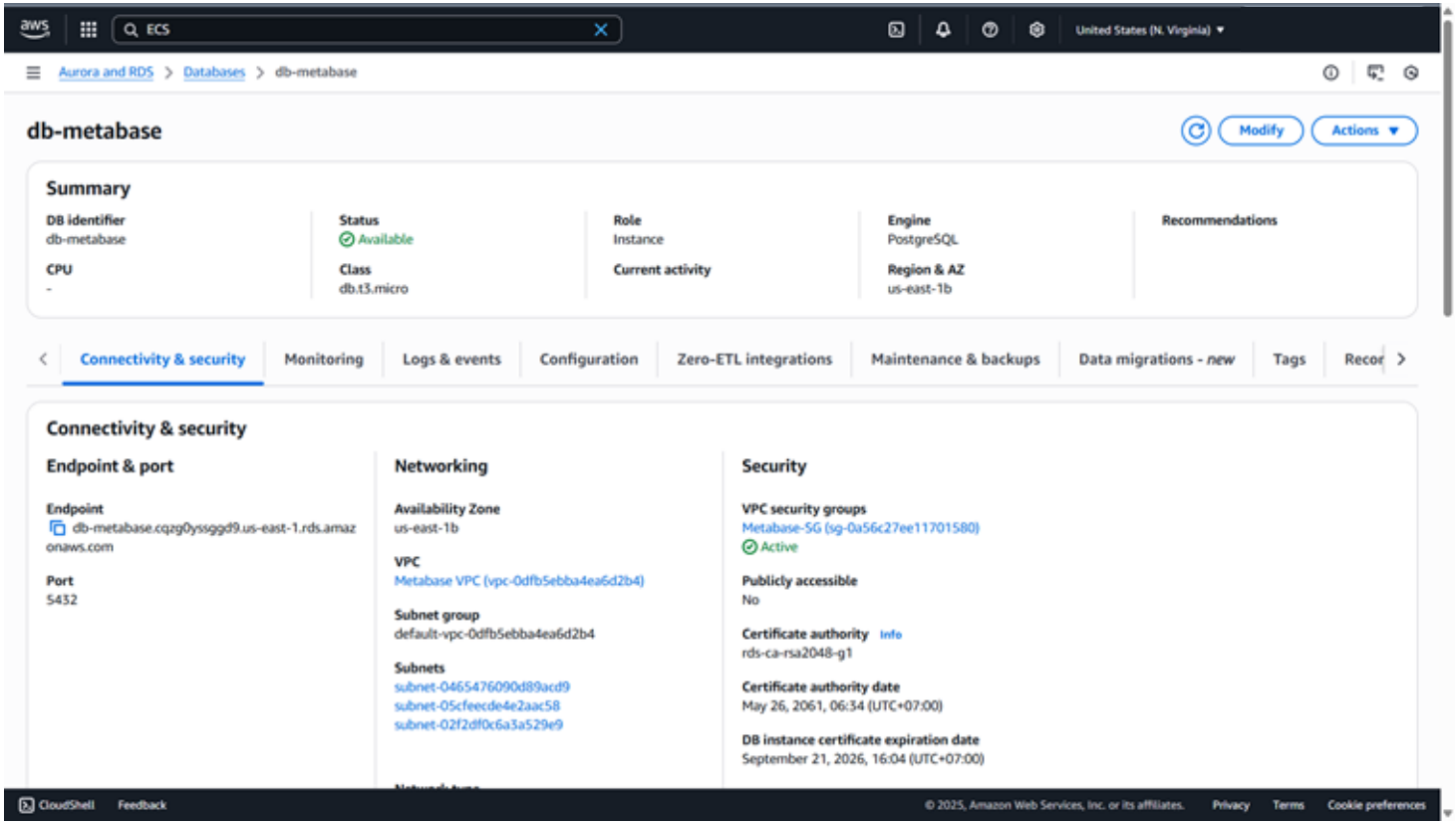
A VPC with public and private subnets was created and an Internet Gateway attached. Route tables and security groups for ALB, ECS tasks, and RDS were configured.

Security Group Traffic Control



Security Groups successfully created and configured. These stateful, virtual firewalls enforce the principle of least privilege, strictly controlling traffic flow. Rules permit external access to the ALB, load balancer traffic to the ECS Tasks, and application traffic to the backend RDS database.

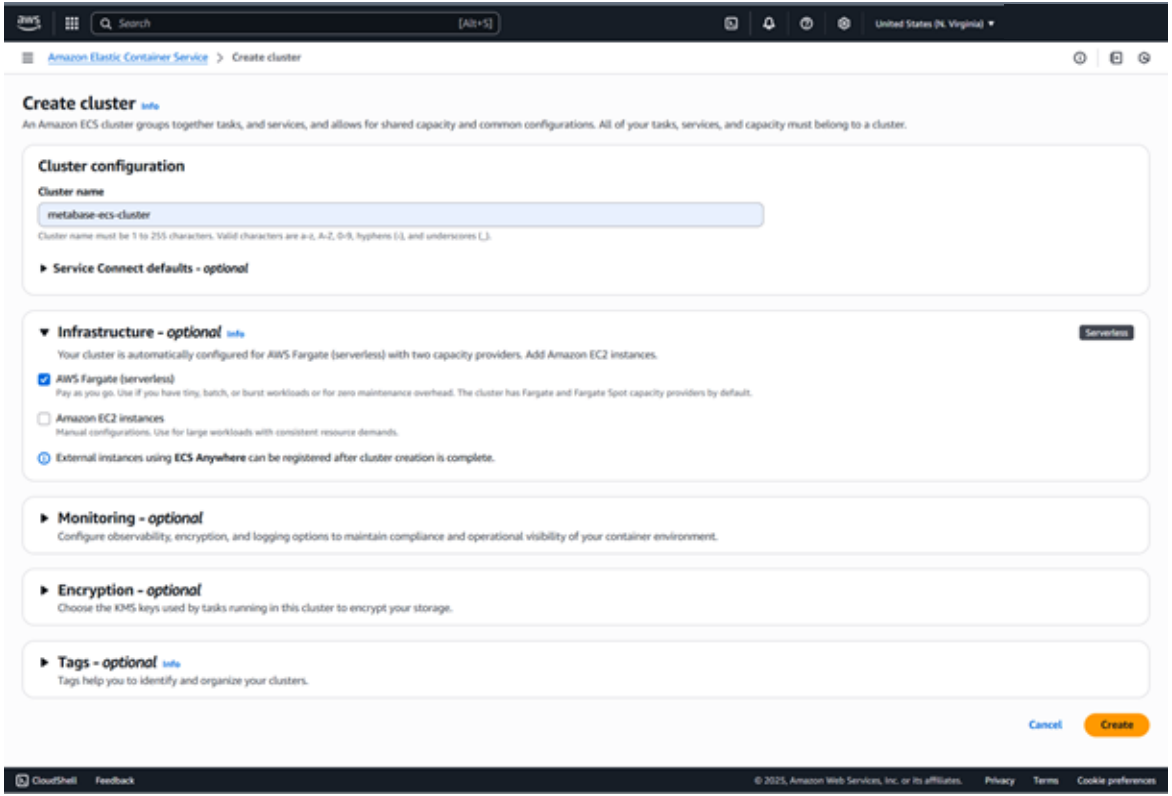
Managed Data Layer: Launching the PostgreSQL RDS



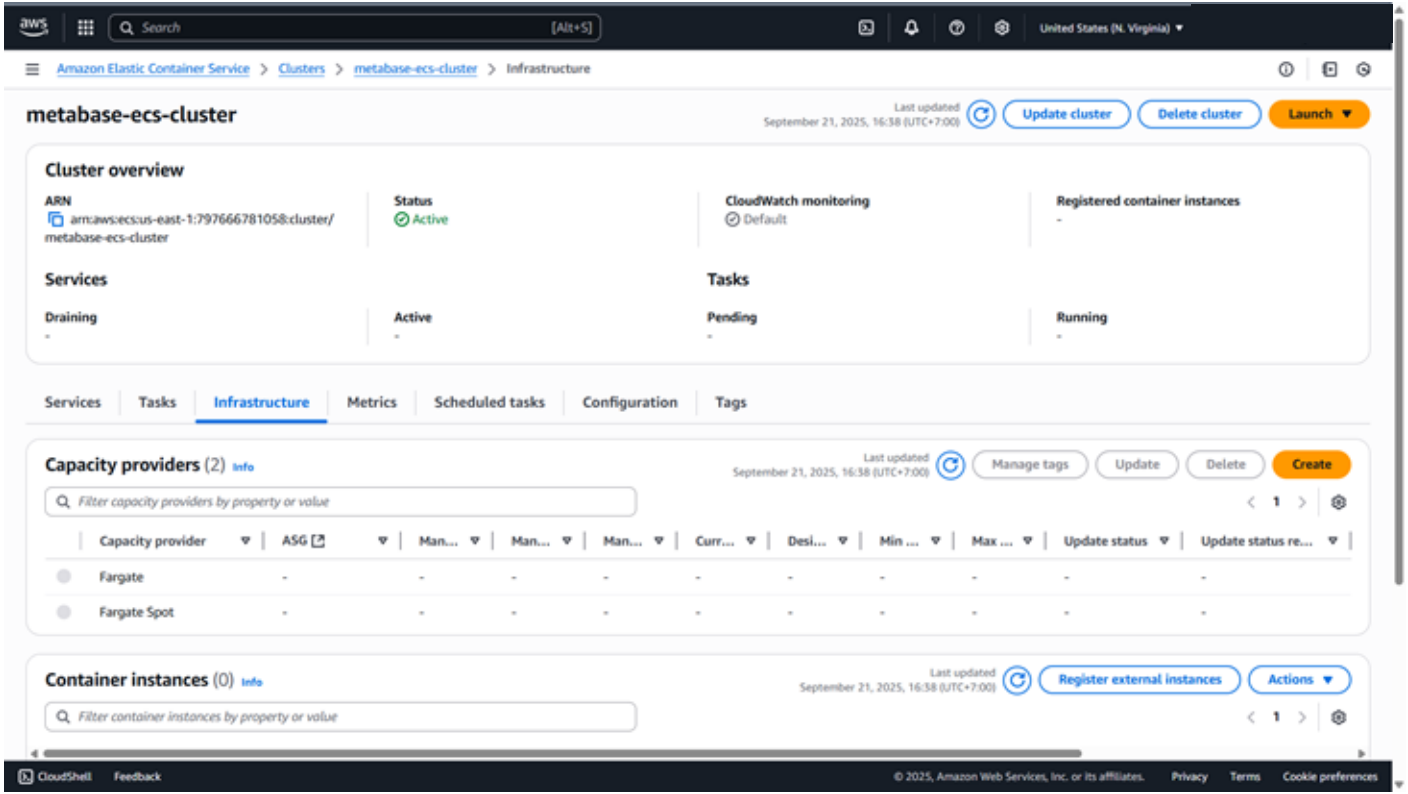
A PostgreSQL RDS instance metabase-db was launched in private subnets. The endpoint and credentials were recorded.

Creating the ECS Fargate Cluster

On the Create Cluster page in the ECS console, the cluster type Networking only (Fargate) was selected and a cluster named metabase-fargate-cluster was created with the default VPC and networking configuration.



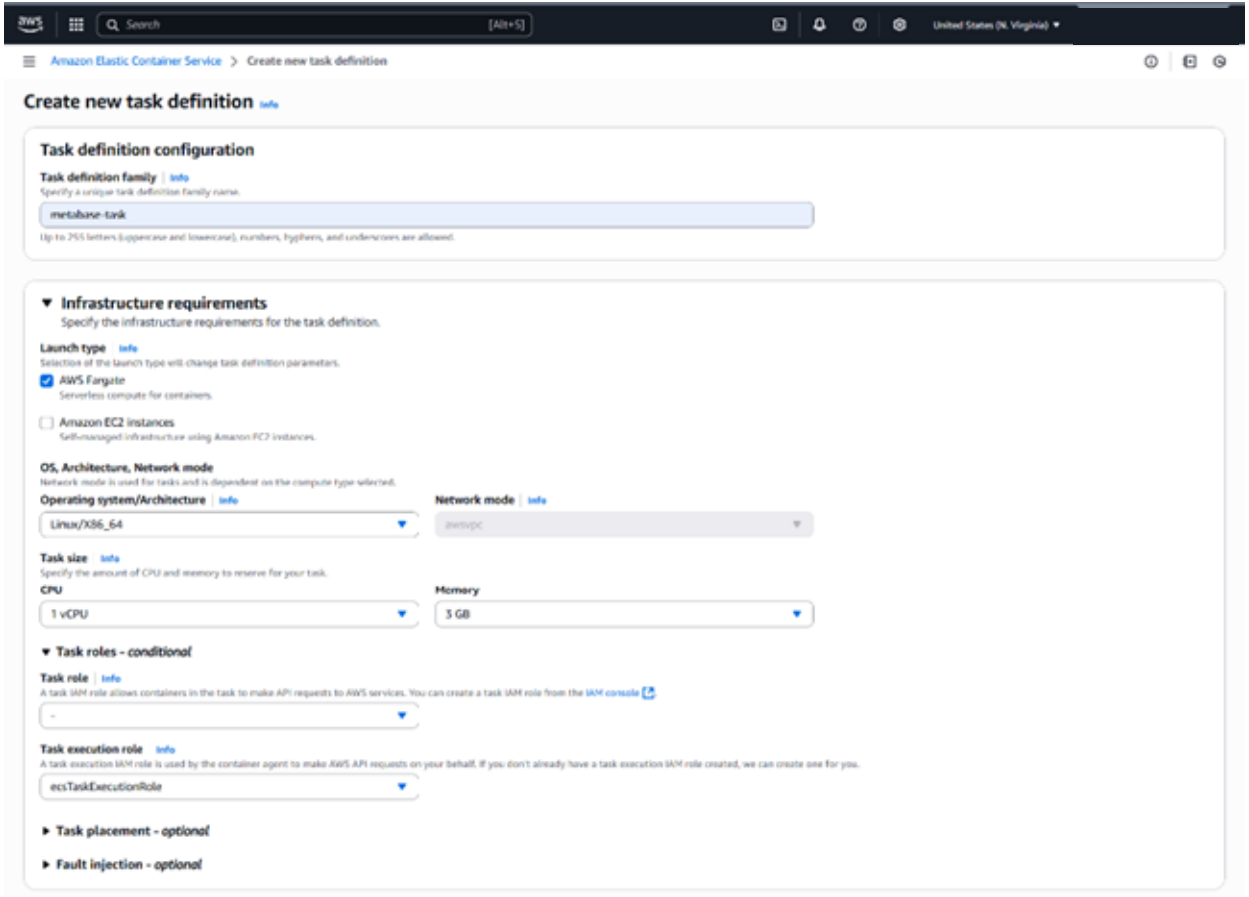
Creating the ECS Fargate Cluster



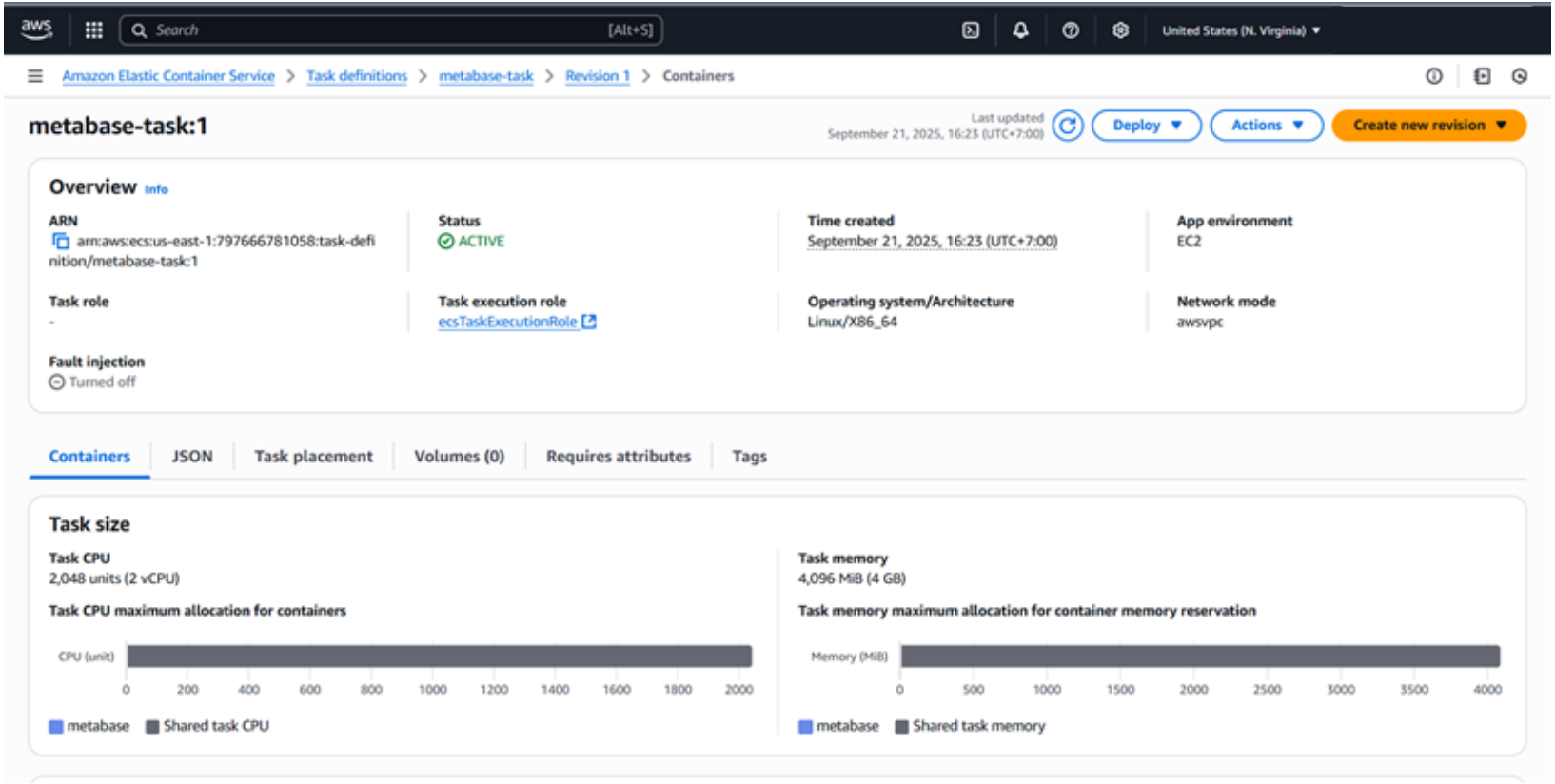
The Cluster Details page now highlights the cluster in an Active state, showing that it’s fully set up and ready to deploy Fargate services without any further configuration needed.

Defining the Task: Container Specs and Settings

On the Create Task Definition page, a new Fargate task definition named metabase-task was set up with 1 vCPU, 3GB memory, and a container using the metabase/metabase:latest image mapped to port 3000. Environment variables for the PostgreSQL database connection were also entered.



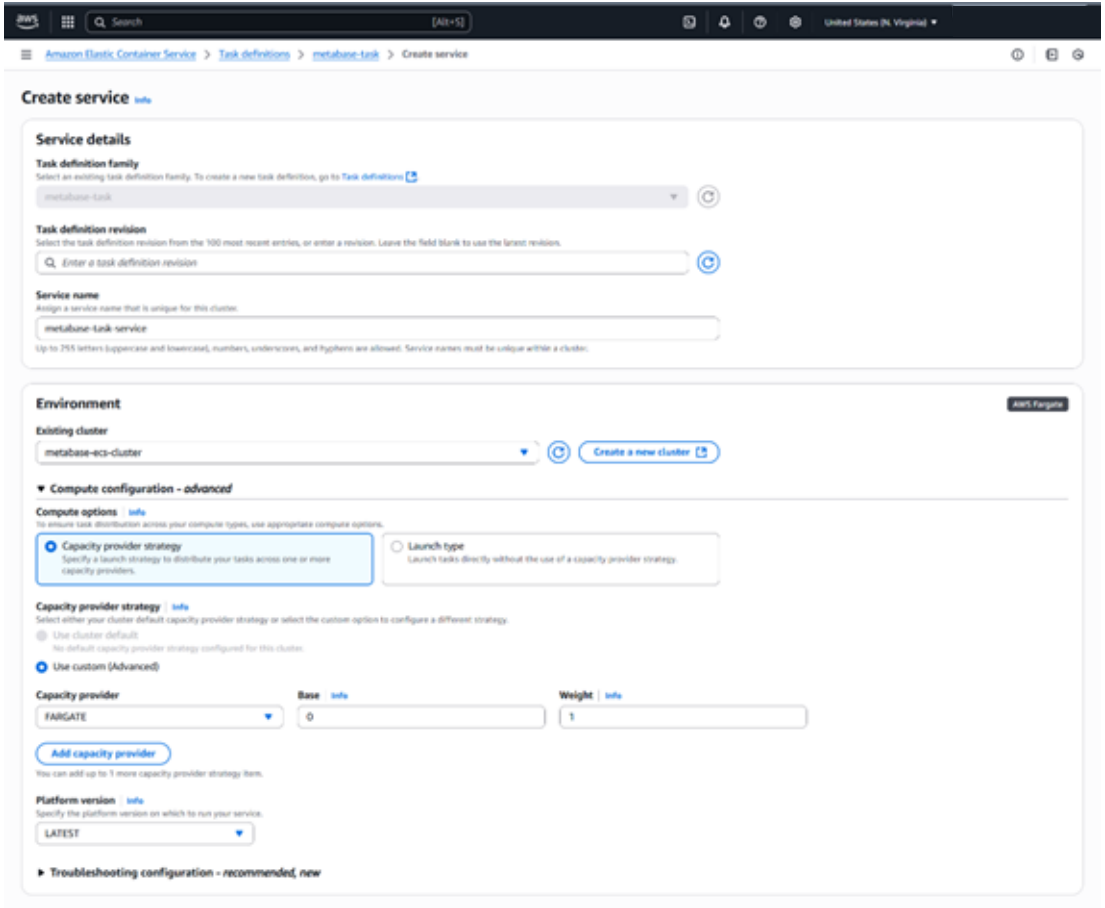
Task Registered



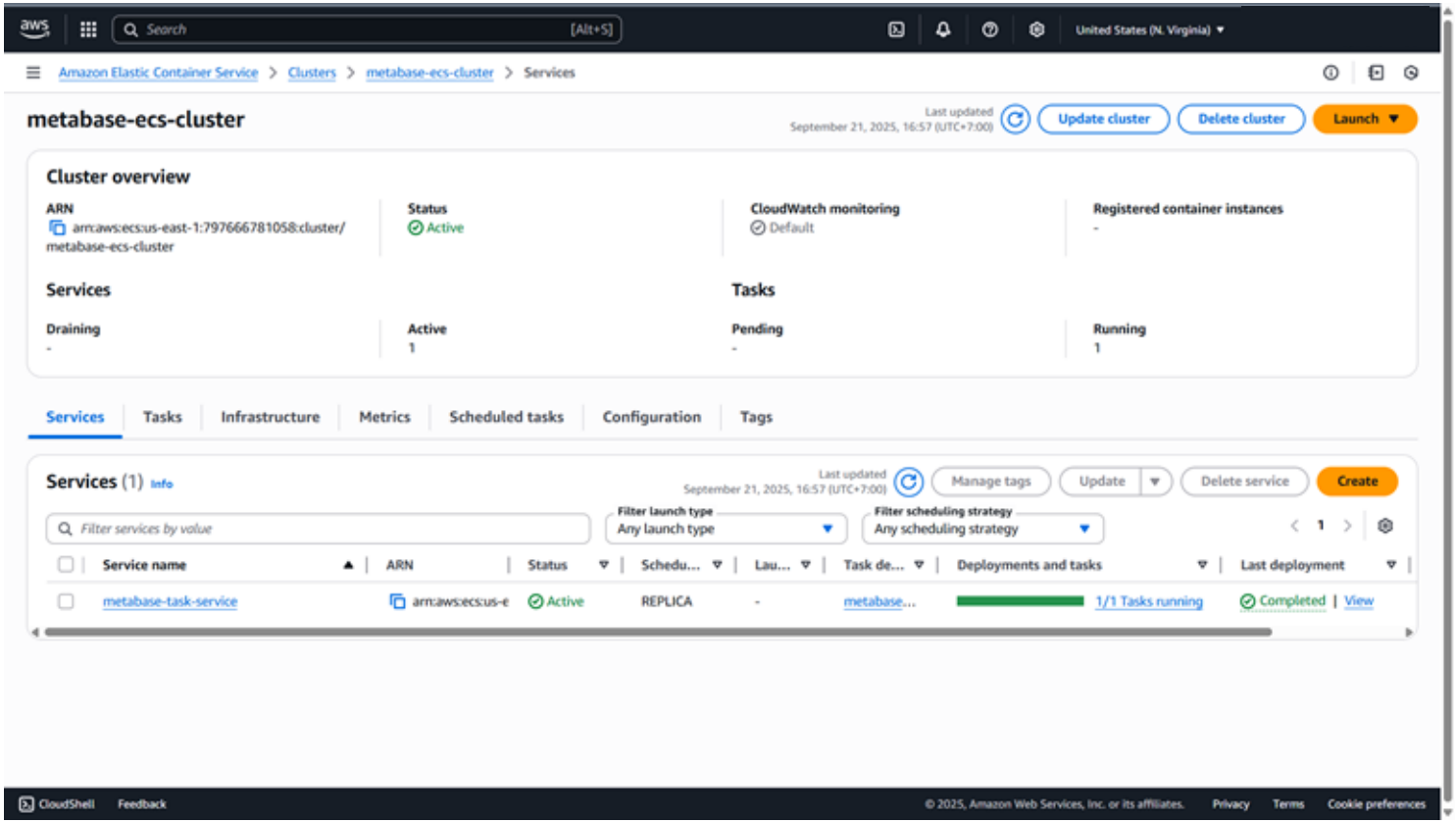
The task definition was successfully registered and is now available as an active revision, ready for deployment of the Metabase container.

Creating the Service

On the Create Service page, an ECS service named metabase-service was configured to run one task in the private subnets. The ECS task security group and target group for traffic forwarding were also attached.

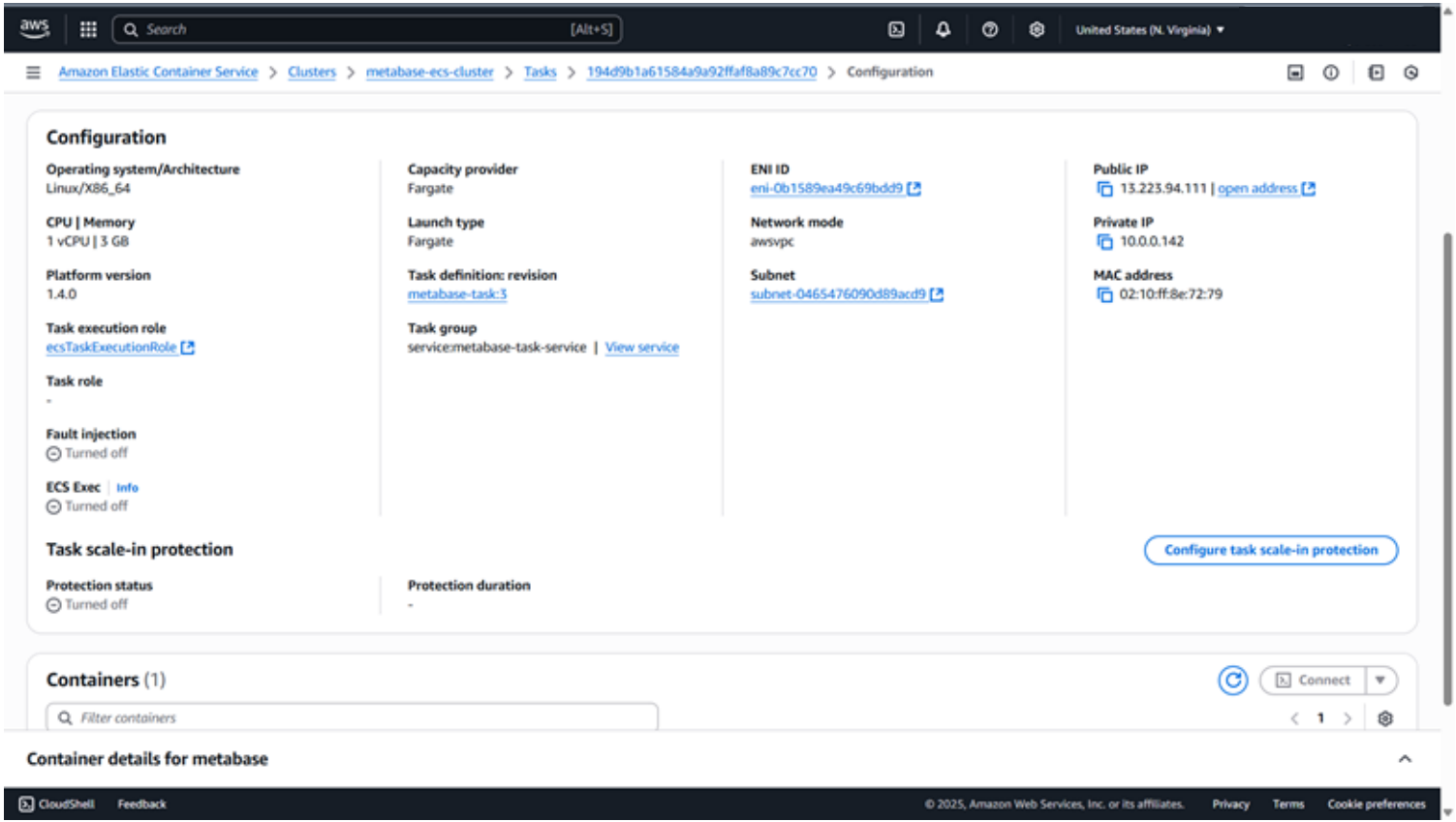


Service Running



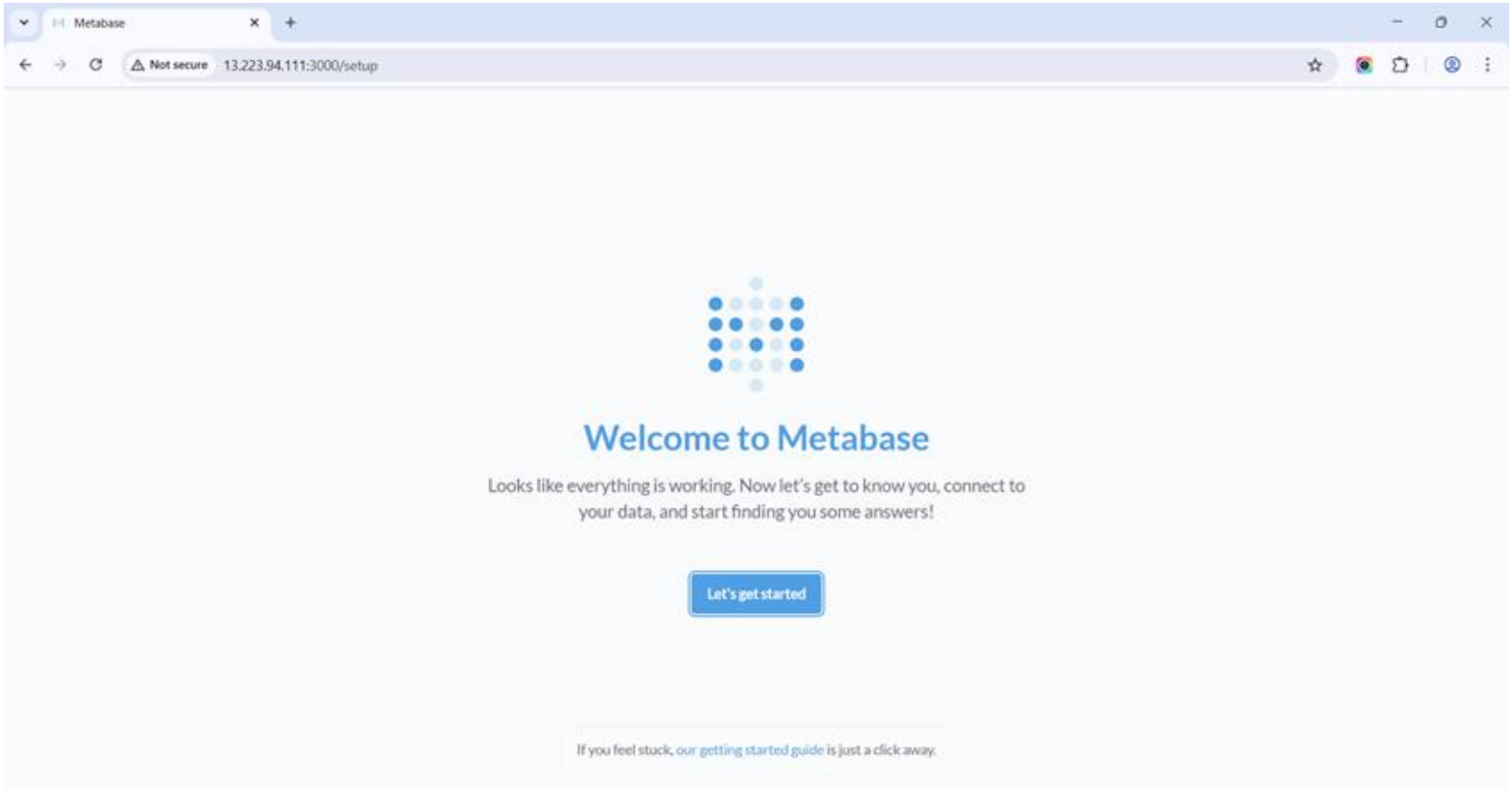
The Service Details page shows the service in a Running and Stable state, with the status marked as Active, confirming that traffic is now seamlessly routed to the ECS tasks through the load balancer.

Accessing the Service via Public IP



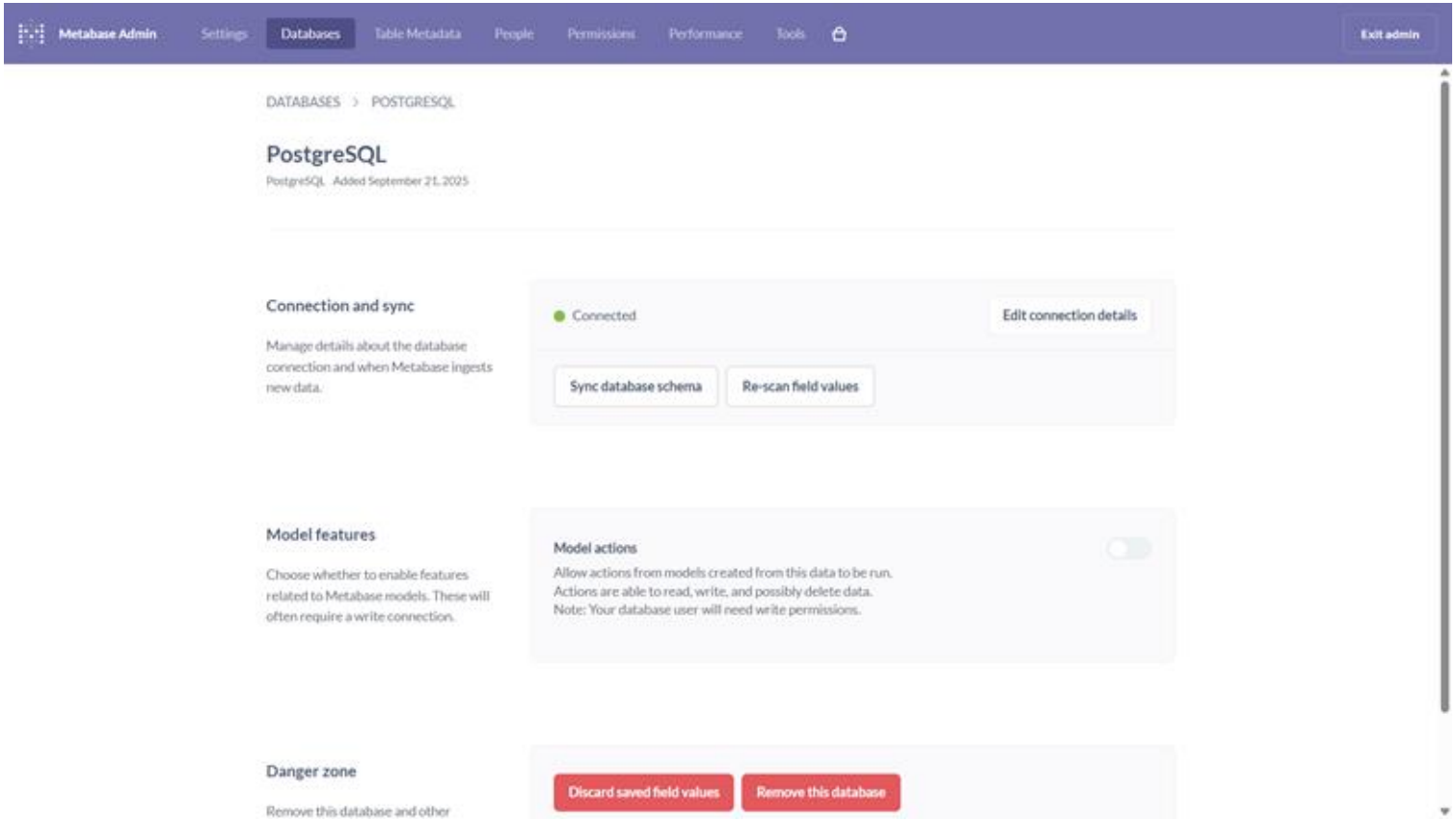
Confirmation of the successful launch of the ECS Fargate task, showing key details such as task ID, public and private IP addresses (if assigned.

Seeing It in Action: Accessing Metabase



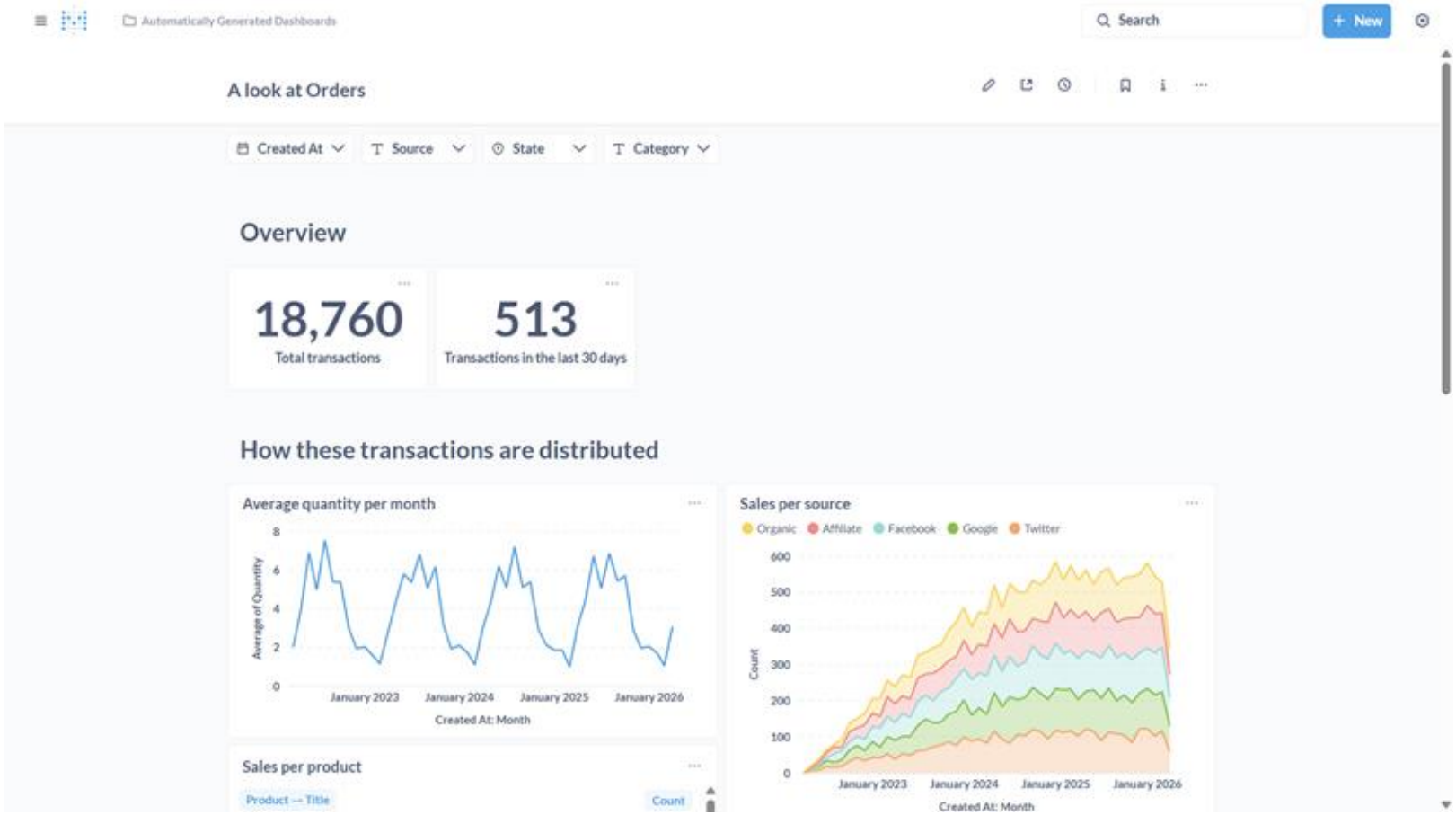
Open this URL in a web browser to confirm the application is accessible and running correctly on the deployed Metabase.

Database Check: Verifying PostgreSQL Connectivity



Confirm that Metabase successfully connects to the PostgreSQL RDS

Sample Metabase Dashboard



Best Practices for Production Deployment

- For a production environment, it is strongly recommended to use an **Application Load Balancer (ALB)** to ensure high availability, automatic traffic distribution, and easier scaling of ECS tasks.
- ALB provides built-in health checks and better traffic routing, which improves reliability and performance compared to exposing ECS tasks directly.
- Although this setup was tested on a **free tier account** where ALB is **not fully supported**, following the ALB-based architecture remains the industry best practice for secure and scalable deployments.
- If free tier limitations are a concern, consider upgrading the AWS account or using a paid plan to leverage ALB features for long-term stability.

Lessons from the Deployment Journey

- A complete Metabase environment was successfully deployed on AWS using **ECS Fargate** and **PostgreSQL RDS**, with a network architecture designed for both scalability and security.
- The environment follows a production-ready design, separating public and private subnets and implementing security groups for controlled access.
- While the current free tier environment cannot fully implement ALB, the architecture has been built with ALB integration in mind, ensuring an easy upgrade path to a production-ready setup when moving to a paid plan.
- This approach provides a solid foundation to scale, secure, and manage Metabase in a cloud-native infrastructure.

Thank You